CHAPTER 1

INTRODUCTION

1.1 Problem Statement

Manual complaint management in educational institutions is inefficient, time-consuming, and lacks transparency. Students face challenges in submitting complaints, tracking their progress, and receiving timely resolutions. A paper-based system results in loss of records, delayed responses, and a lack of accountability. Furthermore, students often have no means to check the status of their complaints, leading to frustration and dissatisfaction. An automated system ensures streamlined complaint handling with accountability, transparency, and efficient resolution tracking.

1.2 Project Aim

The primary objective of this project is to develop an efficient and transparent complaint management system that allows students to submit complaints, track their status, and receive timely responses from administrators. The system aims to eliminate inefficiencies in traditional complaint handling processes by providing an online platform for streamlined communication between students and administrators. The platform will ensure accountability by maintaining a structured database of complaints, tracking updates in real-time, and generating reports for management.

1.3 Scope

- **For Students**:
  - An easy-to-use platform to submit complaints and receive updates.
  - Ability to track the status of complaints in real-time.
  - Notification alerts for complaint progress and resolutions.
- **For Administrators**:
  - A centralized dashboard to manage and categorize complaints.
  - Tools to prioritize urgent complaints and assign them to appropriate staff.
  - Automated complaint resolution tracking and report generation.
- **For Institutions**:
  - A system to enhance transparency and communication.
  - Data analytics to assess recurring issues and improve institutional policies.
  - Secure storage of complaint records for future reference and auditing.

1.4 Technologies Used

- **Frontend**:
  - HTML: Structure of the web interface.
  - CSS: Styling for an intuitive user experience.

- JavaScript: Enhancing interactivity and real-time updates.

- **Backend**:
  - PHP: Server-side scripting to handle business logic.
  - REST API: Connecting the frontend with the backend efficiently.
- **Database**:
  - MySQL: Relational database to store complaint records securely.
  - SQL Queries: To retrieve and manage complaint data efficiently.
  -
- **Additional Tools**:
  - AJAX: Enabling asynchronous requests for a smooth user experience.
  - Email Notifications: Automated alerts to keep students and admins updated.
  -

1.5 SOFTWARE AND HARDWARE REQUIREMENTS

This section describes the software and hardware requirements of the system.

SOFTWARE REQUIREMENTS

OperatingSystem:

- Windows 10 or Linux (Ubuntu recommended) is used as the operating system. Both are stable and provide a robust environment for development. Windows 10 is known for its userfriendliness, while Linux offers better performance for server-side applications and is widely used in web development.

- **WebScrapingTools**:

- **Bright Data** (formerly Luminati) is used for web scraping. It offers a powerful proxy network to scrape data from websites without getting blocked. **Cheerio** is used alongside to parse HTML and traverse the DOM efficiently, making the data extraction process seamless.

- **Database**:

- **MongoDB** is used as the database for storing product price data. MongoDB is a NoSQL database known for its scalability, flexibility, and ability to handle large volumes of data, making it ideal for storing dynamically changing price data.

- **Development Tools and Programming Languages**:
  - **Frontend**: **HTML** and **CSS** are used to build the structure and style the website. HTML provides the skeleton, and CSS is used to create a visually appealing and responsive design.
  - **Frontend Frameworks**: **Node.js** is used for the backend of the frontend, allowing us to handle HTTP requests and server-side logic efficiently. **React.js** is used for creating dynamic, interactive user interfaces, while **TypeScript**

enhances the development experience by adding static types, improving code quality and maintainability. ○ **Backend**: The backend logic and server setup are managed using **Node.js**, ensuring fast and efficient processing of requests, especially for real-time data handling.

- **Libraries and Tools**: **Axios** (for HTTP requests), **Express.js** (for server-side routing), and **Mongoose** (for MongoDB interaction) are used for various backend functionalities.

HARDWARE REQUIREMENTS

- **Processor**:
- **Intel i5** or **Ryzen 5** is recommended as the processor. These processors offer sufficient power for running development tools, handling large-scale web scraping, and ensuring smooth operation of the website during development and testing.

- **RAM**:
  8GB RAM is the minimum requirement. It ensures smooth multitasking and efficient handling of large datasets during web scraping and real-time price tracking.
- **Storage**:
  A minimum of **256GB SSD** is recommended for faster read and write speeds, especially when storing large volumes of price data over time.

- **Internet**:
  A stable and high-speed internet connection is required for web scraping activities to ensure reliable access to external websites for real-time data retrieval.

1.6 Proposed System: Student Complaint Management System

The proposed system aims to streamline the process of managing student complaints, ensuring that student grievances are efficiently collected, tracked, and resolved. The system will provide a platform where students can submit complaints, track their status, and receive feedback, while administrators can monitor, manage, and respond to these complaints effectively.

Objectives:

- **To provide a user-friendly interface** for students to submit complaints.
- **To automate the complaint management process** for administrators, allowing them to track and manage complaints from submission to resolution.
- **To ensure transparency** in the complaint handling process with real-time updates for students.
- **To categorize complaints** based on their type (e.g., academic, infrastructure, staff, etc.) to streamline the resolution process.
- **To ensure timely resolution** and accountability for handling complaints.

Features of the Proposed System:

1. **Student Login and Registration:**
   - Students will need to create an account or log in to access the complaint submission system.

- The system will store user details, such as name, student ID, and contact information, for easy identification and follow-up.

2. **Complaint Submission:**
   - Students can submit complaints through a form, detailing the nature of the issue, including description, category (e.g., academics, facilities, faculty, etc.), and urgency.
   - Students can upload attachments (such as screenshots, documents) for better context.
   - A unique complaint ID will be generated for each submission for tracking purposes.
3. **Complaint Categorization:**
   - Complaints will be categorized automatically based on keywords or by the student selecting the category from a dropdown list (e.g., academic, infrastructure, discipline, etc.).
   - The system will prioritize complaints based on urgency and nature.
4. **Complaint Status Tracking:**
   - Students will be able to view the status of their complaint in real-time (e.g., pending, in progress, resolved).
   - Notifications will be sent to students when there is any change in the status or when feedback is added.
5. **Administrator Dashboard:**
   - Administrators will have a comprehensive dashboard to view all complaints, filter by status, category, or priority, and manage the resolution process.
   - Admins can assign complaints to the relevant department or personnel for resolution.
   - Administrators will also have the ability to add comments or feedback to complaints for better communication with students.
6. **Complaint Resolution:**
   - Once an administrator resolves a complaint, they will update the status, and the student will be notified.
   - The system will track the time taken for resolution and generate reports for performance analysis.
7. **Feedback and Rating:**
   - After resolution, students will be prompted to provide feedback on the handling of their complaint.
   - Students can rate the resolution process, allowing the system to collect valuable insights on the effectiveness of the complaint management process.
8. **Reports and Analytics:**
   - The system will generate reports based on complaint data, such as the number of complaints by category, time taken to resolve complaints, and student satisfaction ratings.
   - These reports will help administrators assess trends, identify recurring issues, and improve the overall complaint management process.
9. **Notifications and Alerts:**
   - Students will receive email and in-app notifications about their complaint status, feedback requests, and any updates.

- Admins will receive alerts about new complaints, pending resolutions, and upcoming deadlines for complaint handling.

10. **Security and Privacy:**

- The system will ensure that all user data, including personal details and complaint information, is securely stored and handled.
- Access control will be implemented to ensure that only authorized personnel can view or manage certain complaints.

Technology Stack:

- **Frontend:** HTML, CSS, JavaScript (for a responsive and interactive UI)
- **Backend:** Python (Flask/Django), Node.js, or PHP (for server-side logic)
- **Database:** MySQL or PostgreSQL (for storing student data, complaints, and resolution statuses)
- **Authentication:** JWT or OAuth for secure login and session management
- **Notifications:** Email notifications using SMTP or third-party services like Twilio or Firebase for real-time notifications

Expected Benefits:

- Improved efficiency in handling student complaints.
- Enhanced communication between students and administrators.
- Better accountability and transparency in the complaint resolution process.
- Data-driven insights for continuous improvement in student services.

CHAPTER 2

REPRESENTATION

2.1 EXPLANATION

The **Student Complaint Management System** is a web-based application designed to help educational institutions efficiently manage and resolve student complaints. This system provides an automated platform where students can submit complaints, track their progress, and receive feedback, while administrators can monitor, assign, and resolve these complaints. By streamlining the complaint handling process, the system ensures that student grievances are addressed promptly and transparently.

Problem Statement:

In many educational institutions, the process of managing student complaints is often manual, disorganized, and slow. Students may feel frustrated due to delayed responses, lack of transparency, or difficulty in tracking the progress of their complaints. The absence of an efficient system can lead to dissatisfaction and unresolved issues, negatively affecting the learning environment. Therefore, there is a need for a centralized, user-friendly system that can handle complaints efficiently and ensure timely resolution.

2.2 Purpose of the System

The purpose of the **Student Complaint Management System** is to automate and streamline the entire process of complaint submission, management, and resolution. The system aims to:

- Provide an easy-to-use platform for students to submit complaints.
- Enable administrators to manage, track, and resolve complaints in a timely manner.
- Enhance transparency by providing students with real-time updates on their complaints.
- Improve the overall quality of services and address recurring issues by analyzing complaint data.

Key Stakeholders

- **Students:** The primary users who submit complaints and provide feedback after the resolution.
- **Administrators:** The secondary users who manage the complaints, assign tasks, and resolve issues.
- **Support Staff/Departments:** Staff members who may be assigned to handle specific complaints, such as academic, infrastructure, or administrative issues.

2.3 System Workflow

1. **Student Registration and Login:** Students create an account or log in using their credentials to access the complaint system.
2. **Complaint Submission:** Students submit complaints, detailing the issue, selecting the category, and adding any relevant attachments (if needed).
3. **Complaint Management (Admin):** Administrators view and manage complaints, categorizing them and assigning them to relevant departments or personnel for resolution.
4. **Resolution and Feedback:** Once resolved, the administrator updates the status of the complaint, and the student is notified. The student is then prompted to provide feedback on the resolution process.
5. **Tracking and Analytics:** The system tracks all complaints, generating real-time reports and insights on the effectiveness of the complaint resolution process.

2.4 Benefits of the System

- **Efficiency:** The system automates complaint handling, reducing manual work and speeding up the resolution process.
- **Transparency:** Students can track the progress of their complaints in real time and receive notifications about any updates or actions taken.
- **Accountability:** Administrators are held accountable for resolving complaints, with each action recorded in the system.
- **Data Insights:** The system generates reports that provide insights into trends, common issues, and areas for improvement, allowing the institution to enhance its services.
- **Improved Communication:** The platform fosters better communication between students and administrators, ensuring that grievances are addressed promptly

2.5 Use case diagram

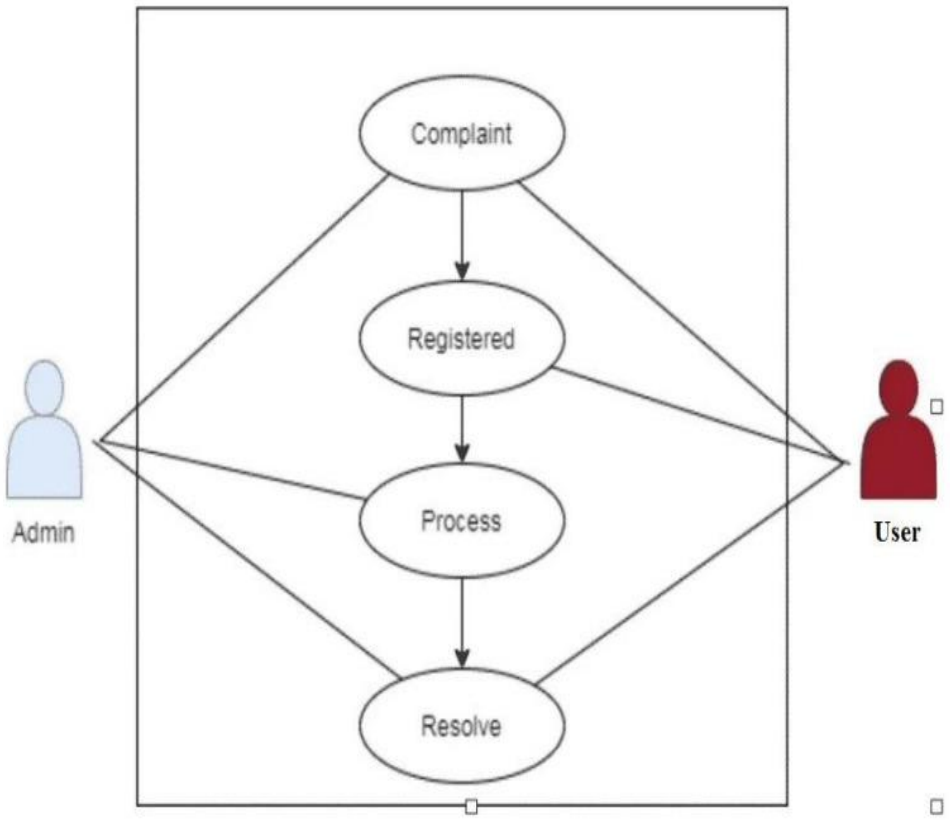


Fig.2.1 Use case diagram

CHAPTER 3

SOFTWARE DESIGN

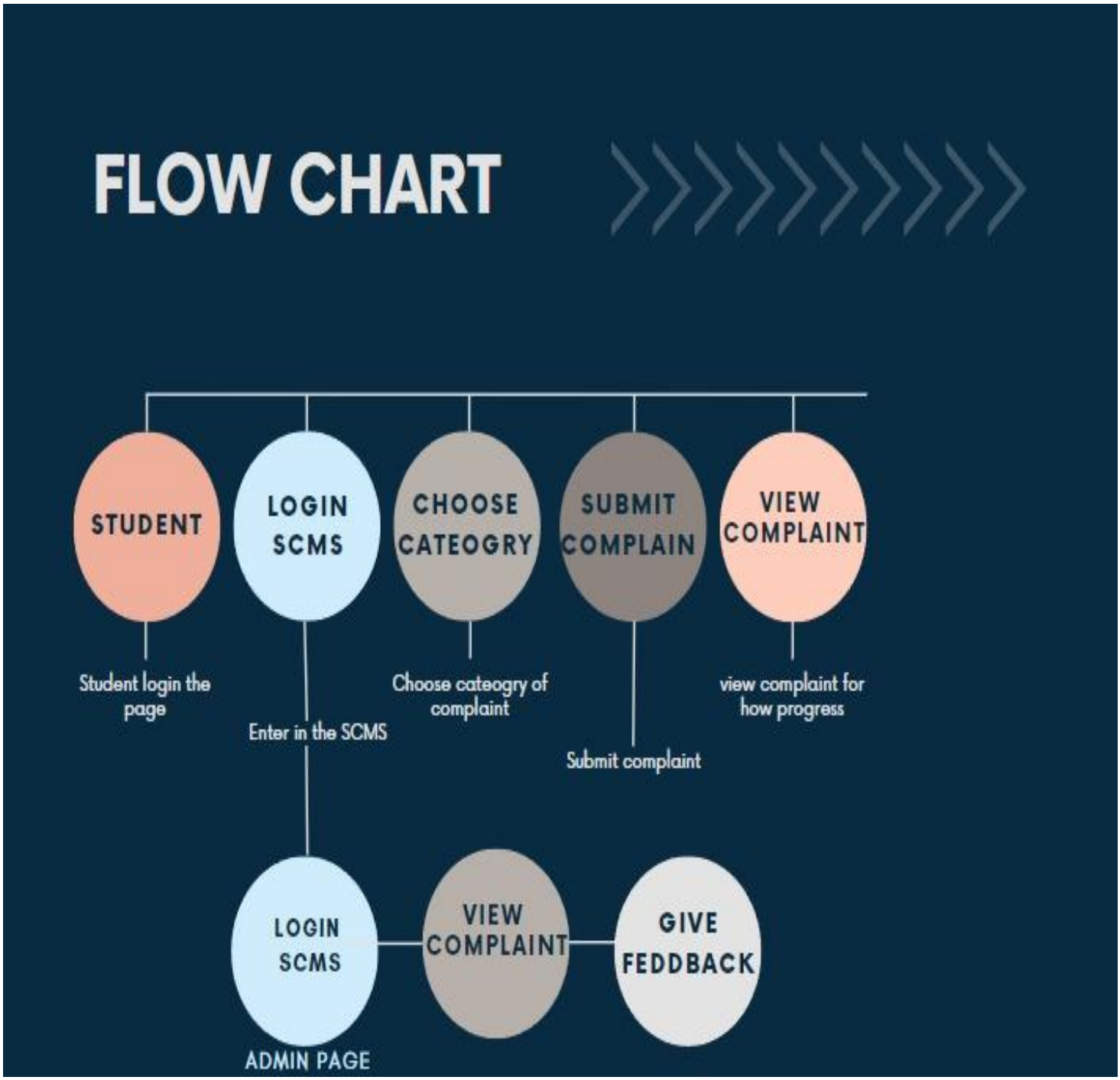


Fig.3.1 Data flow diagram

CHAPTER 4

DATABASE DESIGN

4.1 ENTITY RELATIONSHIP DIAGRAM (ER diagram or ERD):

An ER diagram visually represents the relationships between different data sets within the system. It helps in understanding how different entities (such as tables) are related to one another through various relationships, such as one-to-one, one-to-many, or many-to-many. By mapping out entities and their relationships, we can design a database that is both efficient and structured.
Key database design principles ensure that:

- **Data Integrity**: The consistency and accuracy of data within the system.
- **Data Security**: Protection against unauthorized access and alterations to sensitive student complaint data.
- **Database Flexibility**: The ability of the database to evolve with changing requirements.
- **Database Performance**: Optimizing data retrieval speed and processing efficiency for handling student complaints effectively.

4.2 Designing the Student Complaint Management Database Model:

In the Student Complaint Management System, the database structure is crucial for organizing and storing complaint data efficiently. This involves defining entities (such as Students, Complaints, and Staff), establishing their relationships, and ensuring that the data is stored in a normalized manner for data integrity. While normalization ensures a well-organized structure, denormalization may be used strategically for improved performance in high-traffic scenarios.

4.3 Entities/Tables:

The main entities involved in this system are:

1. **Students**: This table stores details about the students who submit complaints. Each student has a unique ID, and we store their personal information, such as name, email, contact number, and course details.
2. **Complaints**: The core of the system, this table captures the student complaints. Each complaint has a unique ID and records essential information, such as the complaint type, description, submission date, status (open, resolved), and associated student ID.
3. **Staff**: This table holds information about the staff members assigned to resolve complaints. Each staff member is uniquely identified, and their contact information and role (administrator, support staff, etc.) are stored.
4. **Complaint Status History**: To track the status of complaints over time, this table logs the changes in complaint status (from "open" to "in-progress" to "resolved"), the date of each status change, and the staff responsible for each update.

4.4 Relationships:

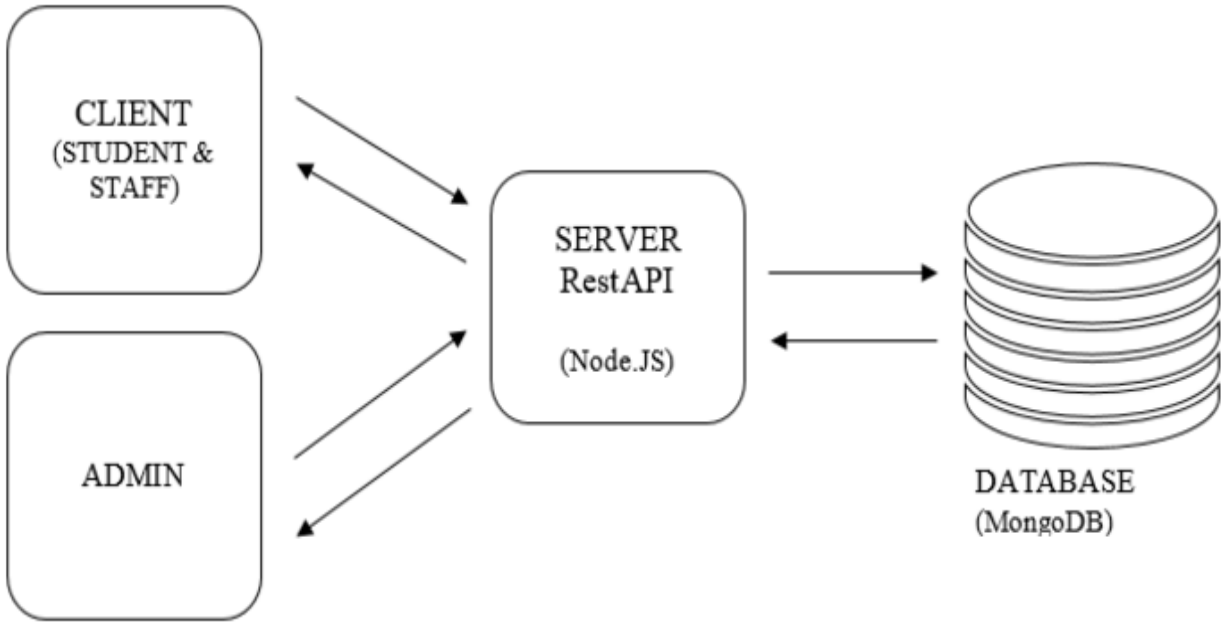
The relationships between the entities ensure data consistency and connectivity across the system. Here are the key relationships:

- **Students to Complaints (One-to-Many)**: A single student can file multiple complaints, but each complaint is linked to only one student. The Student_ID in the Students table serves as the primary key (PK) and is referenced as a foreign key (FK) in the Complaints table.
- **Complaints to Complaint Status History (One-to-Many)**: Each complaint can have multiple status updates (e.g., changing from "open" to "resolved"), but each status update is associated with only one complaint. The Complaint_ID in the Complaints table is referenced as the PK and is linked to the Complaint_Status_History table via a

---

Normalization Principles:

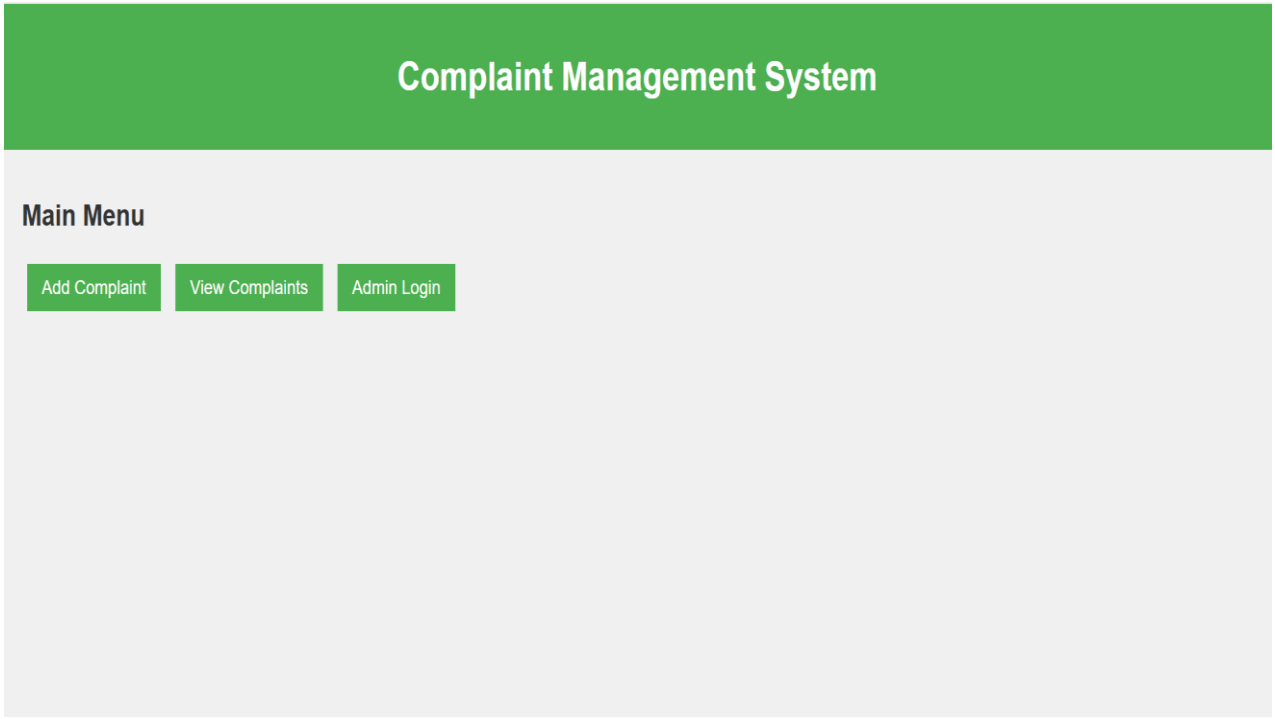
1. **First Normal Form (1NF)**: Ensures that each field contains only atomic (indivisible) values and eliminates duplicate records. For example, the Complaints table ensures that no two complaints have the same ID and that each field (like complaint type and description) stores only a single value.
2. **Second Normal Form (2NF)**: Achieved by removing partial dependencies, ensuring that each non-key attribute depends on the entire primary key. For example, the Complaint Status History table stores records of status updates that are related to a specific complaint ID and cannot be split further into smaller sub-parts.
3. **Third Normal Form (3NF)**: Ensures that all attributes in a table are functionally dependent only on the primary key. In this system, all non-key attributes (such as student names and staff roles) depend solely on their respective primary keys (Student_ID and Staff_ID) and not on other attributes.

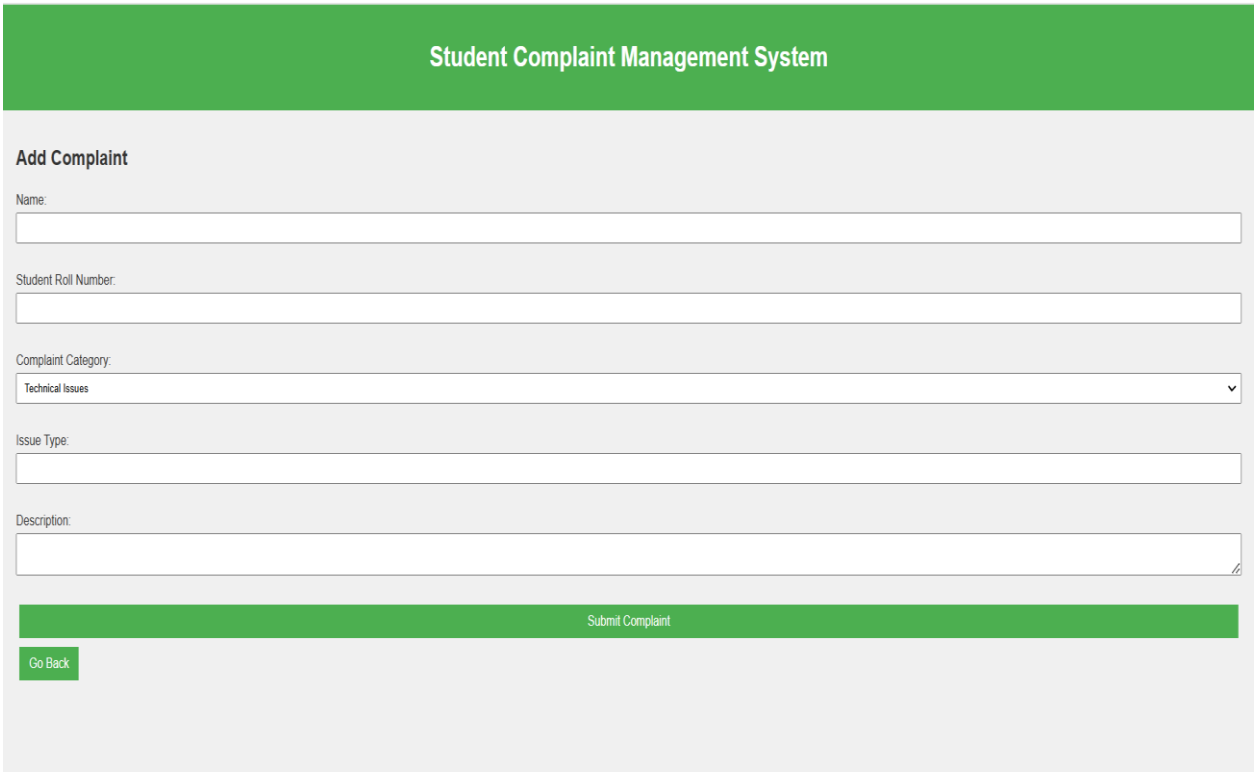


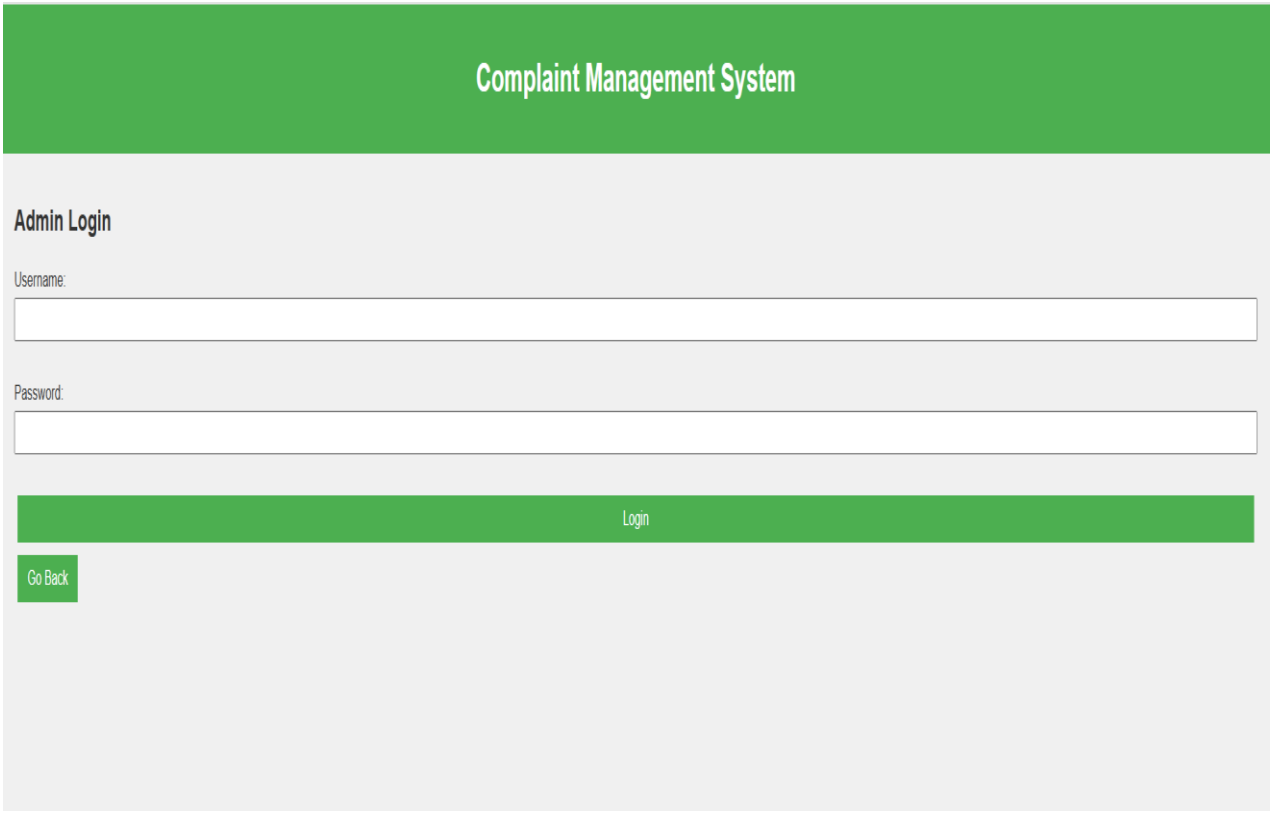
CHAPTER 5

TESTING

5.1 Test cases snapshots






Complaint Management System
Admin Login
Username:
Password:
Login
Go Back


Complaint Management System
Admin Menu
View Complaints
Go Back

CHAPTER 6

ROLES AND RESPONSIBILITIES

6.1 Web Scraping and Front-end Development - Ankit Singh

Ankit Singh is responsible for both implementing and managing the **web scraping functionality** and **frontend development** in the **Student Complaint Management System**. His key tasks include:

- **Web Scraping**:
- Developing web scrapers to extract complaint data, feedback, and student sentiments from various educational websites or forums.
- Ensuring efficient data extraction while handling anti-scraping mechanisms.
- Scheduling and automating the scraping process for regular updates.
- Debugging and optimizing scrapers for accuracy and efficiency.
- Handling errors, timeouts, and failed scraping attempts.
- **Front-end Development**:
- Developing responsive and intuitive UI components for complaint submission, status tracking, and feedback.
- Implementing seamless navigation and smooth user interactions across various sections of the system.
- Integrating the frontend with backend APIs for real-time updates on complaint statuses.
- Ensuring a visually appealing, mobile-friendly design using modern technologies like React.js and Tailwind CSS.
- Optimizing performance and enhancing the user experience, ensuring the system is quick and easy to use.

6.2 Database Management - Paramjeet Singh

Paramjeet Singh is responsible for managing the database that stores and retrieves complaint data efficiently. His key tasks include:

- Designing and maintaining the database schema for managing student complaints, feedback,
- and resolution status.
- Ensuring fast and optimized querying for real-time complaint updates and status changes.
- Implementing data indexing and caching strategies to improve system performance.
- Managing data storage, backups, and retrieval of complaint histories for reporting purposes.
- Ensuring data consistency, security, and integrity within the database.

6.3 Backend Development - Ayush Kumar

Ayush Kumar is responsible for backend development in the **Student Complaint Management System**. His key tasks include:

- Designing and implementing server-side logic and APIs for managing complaint data, user authentication, and status updates.
- Integrating the system with external services (if required) for features like email notifications
  or third-party tools.

- Ensuring the backend supports efficient communication between the frontend and database, enabling real-time updates on complaints.
- Implementing secure login, role-based access control, and ensuring that sensitive student information is handled with confidentiality and integrity.
- Optimizing backend performance to handle high traffic, ensuring smooth user interactions.

CHAPTER 7

CONCLUSION

- The **Student Complaint Management System** is a well-structured and automated platform designed to streamline the complaint submission, tracking, and resolution process for students.
- The system integrates web scraping for gathering complaint data from various online educational forums and platforms to improve the student experience.
- Users can submit complaints, track their status in real-time, and receive updates when their issues are resolved, enabling timely interventions.
- The system is built using technologies like Node.js, React.js, TypeScript, and Tailwind CSS to provide a smooth, interactive user experience.
- With the backend infrastructure in place, the system ensures real-time data exchange between the frontend, backend, and database, providing accurate and up-to-date information.
- MongoDB is used for efficient data storage, ensuring fast querying, and seamless complaint tracking.
- The **Student Complaint Management System** improves communication between students and administrators, enhances accountability, and ensures issues are resolved promptly, ultimately improving student satisfaction and the quality of educational services.

Appendices

FRONTEND: HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Complaint Management System</title>
   <link rel="stylesheet" href="1style.css">


</head>
<body>
   <script type="text/javascript" src="script.js"></script>
   <header>
      <h1>Student Complaint Management System</h1>
   </header>

   <main>
      <div id="main-menu">
         <h2>Main Menu</h2>
         <button onclick="showAddComplaintForm()">Add Complaint</button>
         <button onclick="showComplaints()">View Complaints</button>
         <button onclick="showAdminLogin()">Admin Login</button>
      </div>

      <!-- Add Complaint Form -->
      <div id="add-complaint-form" class="hidden">
         <h2>Add Complaint</h2>
         <form id="complaint-form">
            <label for="name">Name:</label>
            <input type="text" id="name" required><br>

            <label for="stud-rollno">Student Roll Number:</label>
            <input type="text" id="stud-rollno" required><br>

            <label for="category">Complaint Category:</label>
            <select id="category" required>
               <option value="Technical Issues">Technical Issues</option>
               <option value="Service Issues">Service Issues</option>
                        <option value="Student Affairs / Raging">Student Affairs /
Raging</option>
               <option value="Administrative Issues">Administrative Issues</option>
               <option value="Infrastructure">Infrastructure</option>
               <option value="Faculty and Staff">Faculty and Staff</option>
```

```
                    <option value="Financial/Payment Issues">Financial/Payment Issues</option>
                <option value="Other">Other</option>
            </select><br>

            <label for="issue-type">Issue Type:</label>
            <input type="text" id="issue-type" required><br>

            <label for="description">Description:</label>
            <textarea id="description" required></textarea><br>

            <button type="submit">Submit Complaint</button>
        </form>
        <button onclick="goBack()">Go Back</button>
    </div>

    <!-- View Complaints -->
    <div id="complaints-list" class="hidden">
        <h2>List of Complaints</h2>
        <ul id="complaints-ul"></ul>
        <button onclick="goBack()">Go Back</button>
    </div>

    <!-- Admin Login -->
    <div id="admin-login-form" class="hidden">
        <h2>Admin Login</h2>
        <form id="admin-form">
            <label for="admin-username">Username:</label>
            <input type="text" id="admin-username" required><br>

            <label for="admin-password">Password:</label>
            <input type="password" id="admin-password" required><br>

            <button type="submit">Login</button>
        </form>
        <button onclick="goBack()">Go Back</button>
    </div>

    <!-- Admin Panel -->
    <div id="admin-panel" class="hidden">
        <h2>Admin Menu</h2>
        <button onclick="viewComplaintsForAdmin()">View Complaints</button>
        <button onclick="goBack()">Go Back</button>
    </div>
</main>

<script src="script.js"></script>
```

```
</body>
</html>
```

CSS

```
body {
    font-family: Arial, sans-serif;
    background-color: #f0f0f0;
    color: #333;
    margin: 0;
    padding: 0;
}

header {
    background-color: #4CAF50;
    color: white;
    padding: 20px;
    text-align: center;
}

main {
    padding: 20px;
}

button {
    background-color: #4CAF50;
    color: white;
    border: none;
    padding: 10px 15px;
    cursor: pointer;
    margin: 5px;
    font-size: 16px;
}

button:hover {
    background-color: #45a049;
}

form {
    display: flex;
    flex-direction: column;
}

label {
    margin: 5px 0;
```

```
}

input, select, textarea {
    padding: 8px;
    margin-bottom: 10px;
    width: 100%;
    box-sizing: border-box;
}

textarea {
    resize: vertical;
}

.hidden {
    display: none;
}

ul {
    list-style-type: none;
    padding: 0;
}

li {
    margin-bottom: 20px;
}

h2 {
    margin-bottom: 20px;
}
```

Js.script

```
function showAddComplaintForm() {
    hideAllSections();
    document.getElementById('add-complaint-form').classList.remove('hidden');
}

function showComplaints() {
    hideAllSections();
    document.getElementById('complaints-list').classList.remove('hidden');
    fetchComplaints();
}

function showAdminLogin() {
    hideAllSections();
    document.getElementById('admin-login-form').classList.remove('hidden');
}
```

```
function hideAllSections() {
    const sections = document.querySelectorAll('div');
    sections.forEach(section => section.classList.add('hidden'));
}

function goBack() {
    hideAllSections();
    document.getElementById('main-menu').classList.remove('hidden');
}

document.getElementById('complaint-form').addEventListener('submit',
function(event) {
    event.preventDefault();
    const name = document.getElementById('name').value;
    const stud_rollno = document.getElementById('stud-rollno').value;
    const category = document.getElementById('category').value;
    const issueType = document.getElementById('issue-type').value;
    const description = document.getElementById('description').value;


    alert("Complaint submitted: " + name);

    goBack();
});

document.getElementById('admin-form').addEventListener('submit', function(event) {
    event.preventDefault();
    const username = document.getElementById('admin-username').value;
    const password = document.getElementById('admin-password').value;

    if (username === 'admin' && password === 'password123') {
        alert("Admin login successful!");
        showAdminPanel();
    } else {
        alert("Invalid credentials");
    }
});

function showAdminPanel() {
    hideAllSections();
    document.getElementById('admin-panel').classList.remove('hidden');
}

function fetchComplaints() {
    const complaints = [

    ];
```

```
    const complaintsList = document.getElementById('complaints-ul');
    complaintsList.innerHTML = '';

    complaints.forEach((complaint, index) => {
        const li = document.createElement('li');
        li.innerHTML = `
            <strong>${complaint.name}</strong> (Roll No: ${complaint.rollno})
            <br>Category: ${complaint.category}
            <br>Description: ${complaint.description}
            <br><button onclick="approveComplaint(${index})">Approve</button>
            <button onclick="rejectComplaint(${index})">Reject</button>
            <button onclick="deleteComplaint(${index})">Delete</button>
        `;
        complaintsList.appendChild(li);
    });
}

function approveComplaint(index) {
    alert("Complaint " + (index + 1) + " approved.");
}

function rejectComplaint(index) {
    alert("Complaint " + (index + 1) + " rejected.");
}

function deleteComplaint(index) {
    alert("Complaint " + (index + 1) + " deleted.");
}
```

Backend:PHP

```
class DatabaseConnection:
    def __init__(self):
        # Simulated database (replace with actual DB connection)
        self.data = []

    def Add(self, name, stud_rollno, complaint_category, issue_type, description):
        # Simulate adding data to the database
        complaint_data = {
            'name': name,
            'stud_rollno': stud_rollno,
            'complaint_category': complaint_category,
```

```
            'issue_type': issue_type,
            'description': description,
            'approved': None,  # None means not approved yet
            'feedback': None    # Feedback from the admin
        }
        self.data.append(complaint_data)
        return "Information saved successfully!"

    def GetComplaints(self):
        # Return the complaints stored in the database
        return self.data

    def DeleteComplaint(self, index):
        if 0 <= index < len(self.data):
            del self.data[index]
            return "Complaint deleted successfully."
        else:
            return "Invalid complaint index."

    def ApproveComplaint(self, index, feedback):
        if 0 <= index < len(self.data):
            self.data[index]['approved'] = True
            self.data[index]['feedback'] = feedback
            return "Complaint approved and feedback provided."
        else:
            return "Invalid complaint index."

    def RejectComplaint(self, index, feedback):
        if 0 <= index < len(self.data):
            self.data[index]['approved'] = False
            self.data[index]['feedback'] = feedback
            return "Complaint rejected and feedback provided."
        else:
            return "Invalid complaint index."

# Admin login function
def admin_login():
    admin_username = "admin"
    admin_password = "password123"

    print("Admin Login")
    username = input("Enter admin username: ")
    password = input("Enter admin password: ")

    if username == admin_username and password == admin_password:
        print("Login successful!")
        return True
    else:
```

```
        print("Invalid credentials. Please try again.")
        return False

# Create a DatabaseConnection instance
conn = DatabaseConnection()

def SaveData():
    # Collect user input from the command line
    name = input("Enter your name: ")
    stud_rollno = input("Enter your student roll number: ")

    print("\nSelect complaint category:")
    print("1. Technical Issues")
    print("2. Service Issues")
    print("3. Student Affairs / R   aging")
    print("4. Administrative Issues")
    print("5. Infrastructure")
    print("6. Faculty and Staff")
    print("7. Financial/Payment Issues")
    print("8. Other")

    # Ensure the user chooses a valid category
    category_choice = input("Enter your choice (1-8): ")

    if category_choice == '1':
        complaint_category = 'Technical Issues'
    elif category_choice == '2':
        complaint_category = 'Service Issues'
    elif category_choice == '3':
        complaint_category = 'Student Affairs/Raging'
    elif category_choice == '4':
        complaint_category = 'Administrative Issues'
    elif category_choice == '5':
        complaint_category = 'Infrastructure'
    elif category_choice == '6':
        complaint_category = 'Faculty and Staff'
    elif category_choice == '7':
        complaint_category = 'Financial/Payment Issues'
    elif category_choice == '8':
        complaint_category = 'Other'
    else:
        print("Invalid choice, setting to 'Other'.")
        complaint_category = 'Other'

    print(f"Selected Category: {complaint_category}")
    issue_type = input("Enter the issue types: ")
    description = input("Enter your complaint description: ")
```

```
    # Save the complaint data to the simulated database
        message = conn.Add(name, stud_rollno, complaint_category, issue_type,
description)
    print(message)

def ShowComplainList():
    complaints = conn.GetComplaints()
    if complaints:
        print("\nList of Complaints:")
        for idx, complaint in enumerate(complaints, start=1):
            approval_status = "Approved" if complaint['approved'] else "Not Approved"
                            print(f"{idx}. Name: {complaint['name']}, Roll Number:
{complaint['stud_rollno']}, Category: {complaint['complaint_category']}")
            print(f"   Issue Type: {complaint['issue_type']}")
            print(f"   Complaint: {complaint['description']}")
            print(f"   Status: {approval_status}")
            if complaint['feedback']:
                print(f"   Admin Feedback: {complaint['feedback']}\n")
            else:
                print(f"   No feedback yet.\n")
    else:
        print("\nNo complaints found.")

def AdminMenu():
    while True:
        print("\nAdmin Menu:")
        print("1. View All Complaints")
        print("2. Approve a Complaint")
        print("3. Reject a Complaint")
        print("4. Delete a Complaint")
        print("5. Exit to Main Menu")

        choice = input("Enter your choice (1, 2, 3, 4, or 5): ")

        if choice == '1':
            ShowComplainList()
        elif choice == '2':
            ShowComplainList()
            try:
                index = int(input("Enter the complaint number to approve: ")) - 1
                feedback = input("Enter feedback for the complaint: ")
                message = conn.ApproveComplaint(index, feedback)
                print(message)
            except ValueError:
                print("Invalid input, please enter a valid complaint number.")
        elif choice == '3':
            ShowComplainList()
            try:
```

```
            index = int(input("Enter the complaint number to reject: ")) - 1
            feedback = input("Enter feedback for the rejection: ")
            message = conn.RejectComplaint(index, feedback)
            print(message)
        except ValueError:
            print("Invalid input, please enter a valid complaint number.")
    elif choice == '4':
        ShowComplainList()
        try:
            index = int(input("Enter the complaint number to delete: ")) - 1
            message = conn.DeleteComplaint(index)
            print(message)
        except ValueError:
            print("Invalid input, please enter a valid complaint number.")
    elif choice == '5':
        break
    else:
        print("Invalid choice, please try again.")

def main():
    while True:
        # Main menu for the Complaint Management System
        print("\nComplaint Management System")
        print("1. Add Complaint")
        print("2. View Complaints")
        print("3. Admin Login")
        print("4. Exit")

        choice = input("Enter your choice (1, 2, 3, or 4): ")

        if choice == '1':
            SaveData()
        elif choice == '2':
            ShowComplainList()
        elif choice == '3':
            if admin_login():
                AdminMenu()
        elif choice == '4':
            print("Exiting the system. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main = main()
```